# Recorrido del código

highOrderManufacturedFDAImplicitPETScDistributed

Generado el 2026-10-18 a partir de
highOrderManufacturedFDAImplicitPETScDistributed.C (5429 líneas) y de los headers
de src/monodomainCommon

**Cómo leer esto.** Los listados NO son copias: salen de \lstinputlisting apuntando al fuente real, con rangos que este reporte recalcula al regenerarse. El PDF de hoy muestra el código de hoy. Los números de línea del margen son los del archivo.

## Índice


**1. Qué resuelve este solver** **2**

**2. De qué se compone: el solver, el código común y una capa** **2**

**3. Andamiaje de PETSc: errores y matriz distribuida** **3**

**4. El offset global de fila** **4**

**5. Nombres de opciones: una sola ortografía, y los de PETSc** **5**

**6. La solución manufacturada** **7**

**7. Bloques de construcción del ensamblado** **9**

**8. Las dos matrices de rigidez** **11**

**9. Resolución del sistema lineal** **13**

**10. Discretización temporal: las dos familias** **16**

**11. El driver de Vm que manejan las EDO** **19**

**12. Reconstrucción a los puntos de integración** **21**

**13. Integración de las EDO, y el paralelismo por hilos** **23**

**14. main(): montaje** **25**

**15.main(): el lazo temporal y los métodos no lineales** **27**

**16.Índice de funciones** **30**

## 1. Qué resuelve este solver

Resuelve la ecuación de monodominio con una **solución manufacturada**: se elige de antemano un campo exacto $V_m(x,t)$ y tres estados $u_1, u_2, u_3$, se los mete en la ecuación, y lo que sobra se agrega como término fuente. Con eso el error es *medible* y no estimable, que es lo que permite demostrar el orden de precisión — el resultado central del solver.

La ecuación discretizada en el tiempo, con $\theta$ eligiendo el esquema ($\theta = 1$ backward Euler, $\theta = 1/2$ Crank–Nicolson), es

$$\left(\tfrac{M}{\Delta t} - \theta\,\lambda K\right) V_m^{n+1} \;=\; \left(\tfrac{M}{\Delta t} + (1-\theta)\,\lambda K\right) V_m^{n} \;+\; b$$

con $M$ la matriz de masa, $K$ el operador de difusión y $b$ el término fuente más las condiciones de borde. El archivo está organizado casi exactamente en ese orden: primero la infraestructura lineal, después $M$ y $K$, después la parte no lineal, y al final el lazo temporal.

**Dos ejes de paralelismo**, y conviene tenerlos separados desde el principio porque el código los trata en lugares distintos: **MPI** reparte la malla entre ranks y toca todas las fases, mientras **OpenMP** reparte, dentro de un rank, la integración de las EDO (por celda o por punto de Gauss), las reconstrucciones en los puntos de Gauss y los productos matriz–vector de Eigen. El ensamblado no tiene hilos.

La ecuación de arriba es la familia $\theta$; los BDF1–4 entran en el mismo par de operadores cambiando sólo el escalar que multiplica a $M$ (sección de discretización temporal).

## 2. De qué se compone: el solver, el código común y una capa

Conviene decirlo antes de mirar un listado, porque los nombres que aparecen en todos ellos no dicen de dónde salen. El solver se construye a partir de estos archivos del repositorio y de nada más:

- `highOrderManufacturedFDAImplicitPETScDistributed.C`: lo propio del MMS — la solución manufacturada y su fuente, el modelo de tres estados, sus EDO, y el lazo temporal;
- `createFields.H`, que lee los diccionarios y construye los campos y los operadores;
- `src/monodomainCommon/*.H` (desde 2026-10): lo que el MMS comparte con el solver electro — el offset global y la validación de opciones (`monodomainBase.H`), el ensamblado de $M$ y $K$ (`monodomainAssembly.H`), el puente con PETSc (`monodomainPetsc.H`), GMRES/JFNK (`monodomainKrylov.H`), los esquemas temporales y el driver de $V_m$ (`monodomainTimeSchemes.H`), las reconstrucciones a los puntos de Gauss (`monodomainReconstruction.H`) y las claves de OpenMP;
- `src/highOrderAdapter/highOrderInterp.H`, la capa de alto orden.

Los headers comunes no son una biblioteca: se incluyen *dentro* del espacio de nombres anónimo del `.C`, así que cada ejecutable compila su propia copia y un cambio ahí cambia los dos solvers. Cada listado de este recorrido indica entre corchetes de qué archivo sale.

Afuera queda OpenFOAM, PETSc, Eigen y **solids4foam**, de donde sale la reconstrucción de alto orden propiamente dicha. Después de sacar del `Make/options` las inclusiones y bibliotecas que ya no se usaban, el binario enlaza tres objetos compartidos: `libsolids4FoamModels`, `libpetsc` y `libOpenFOAM`.

**Por qué el código dice "LRE" en todas partes.** La reconstrucción la hacía originalmente LRE (Castrillo *et al.*, *Comput Struct* 268:106829, 2022), una biblioteca **serial**: su construcción de

stencils retorna antes del intercambio MPI — que está comentado — y los patches de procesador lanzan `NotImplemented`. Es decir que el port a MPI no podía hacerse sobre ella. La reemplaza `movingLeastSquares`, también de solids4foam, que sí es paralela de verdad: los stencils llevan **identificadores globales** de celda y cada punto de evaluación hace su propio intercambio de halo, de modo que el llamador no cambia para correr en paralelo. Los nombres `LREInterp_Vm`, `LREInterp_Iion`, `LREInterp_states` y las claves `LRECoeffs*` del diccionario son **históricos**: LRE ya no interviene.

**Qué hace la capa**, porque casi todo lo que contiene es reenvío de una línea y podría no existir. Cuatro diferencias de contrato la justifican:

1. **Las claves del diccionario.** El solver y todos los casos del árbol de corridas están escritos en la ortografía de LRE (`N`, `Nn`, `weightFunction`, `k`), y la capa las traduce a las de `movingLeastSquares`. Vale la pena saber que `N` es el *grado* del polinomio y `Nn` son las celdas *agregadas* sobre el mínimo, que es el número de términos del desarrollo de Taylor: $(p{+}1)(p{+}2)/2$ en 2D y $(p{+}1)(p{+}2)(p{+}3)/6$ en 3D. Con `N 2` y `Nn 10` en 2D el stencil tiene $6+10=16$ celdas.
2. **Los coeficientes de derivadas por debajo del orden que los necesita.** LRE reservaba los arreglos de derivada segunda y tercera siempre, así que el solver liga la referencia y recién guarda el *uso* con `order() >= 2`. `movingLeastSquares` no los construye por debajo del orden requerido y **aborta** si se los piden; la capa devuelve una lista vacía y eso es lo que mantiene correctos los tres bindings incondicionales del solver.
3. **Los pesos de cuadratura cambiaron de significado.** LRE los normalizaba — una cara sumaba 1 — y el llamador multiplicaba por $|S_f|$; los de solids4foam son **físicos** y suman $|S_f|$. Para que la diferencia no reescalara un resultado en silencio, los accesores se llaman `faceQuadWeightPhysical()` y `cellQuadWeightPhysical()`: un sitio portado sin revisar **no compila**.
4. **Una sobrecarga rota.** La forma de `fGrad` que devuelve `autoPtr` llama a un miembro que no existe, así que la capa dimensiona el resultado y usa la de dos argumentos.

Nada de eso desaparece si se borra la capa: se muda a cada consumidor. Y hay dos consumidores, porque el solver del electro usa la misma.

## 3. Andamiaje de PETSc: errores y matriz distribuida

El solver ensambla en Eigen y resuelve en PETSc, así que lo primero del archivo es el puente entre los dos.

`checkPetscError` convierte un código de retorno de PETSc en un `FatalError` de OpenFOAM con el contexto de la llamada. Sin eso un error de PETSc se propaga como un entero que nadie mira.

`buildDistributedMat` es la pieza central del port a MPI: arma la matriz `MPIAIJ` a partir de los tripletes locales. La matriz es **nLocal** × **nGlobal** — filas propias, columnas globales — que es el layout que PETSc espera. En serie `gRowStart` vale cero y `nLocal == nGlobal`, así que el camino es el mismo.

```
20  void checkPetscError(const PetscErrorCode ierr, const char* context)
21  {
22      if (ierr)
23      {
24          FatalErrorInFunction
25              << "PETSc call failed in " << context
26              << " with error code " << ierr
27              << exit(FatalError);
28      }
29  }
```

Listing 1: Un código de retorno de PETSc pasa a ser un error legible. [monodomainPetsc.H]

```
    void buildDistributedMat(const SpMat& A, Mat& out)
    {
        const PetscInt nLocalRows = static_cast<PetscInt>(A.rows());
        const PetscInt nGlobalCols = static_cast<PetscInt>(A.cols());
        const PetscInt rStart = static_cast<PetscInt>(globalRowStart());
        const PetscInt rEnd = rStart + nLocalRows;

        std::vector<PetscInt> dNnz(nLocalRows, 0);
        std::vector<PetscInt> oNnz(nLocalRows, 0);
        for (PetscInt row = 0; row < nLocalRows; ++row)
        {
            for (SpMat::InnerIterator it(A, row); it; ++it)
            {
                const PetscInt c = static_cast<PetscInt>(it.col());
                if (c >= rStart && c < rEnd)
                {
                    ++dNnz[row];
                }
                else
                {
                    ++oNnz[row];
                }
            }
        }

        checkPetscError(MatCreate(PETSC_COMM_WORLD, &out), "MatCreate");
        checkPetscError
        (
            MatSetSizes
            (
                out,
                nLocalRows,
                nLocalRows,     // local size of the vector this matrix multiplies
                PETSC_DETERMINE,
                nGlobalCols
            ),
            "MatSetSizes"
        );
        checkPetscError(MatSetType(out, MATAIJ), "MatSetType");
        checkPetscError
        (
            MatSeqAIJSetPreallocation
            (
                out, 0, dNnz.empty() ? nullptr : dNnz.data()
            ),
            "MatSeqAIJSetPreallocation"
        );
        checkPetscError
        (
            MatMPIAIJSetPreallocation
            (
                out,
                0, dNnz.empty() ? nullptr : dNnz.data(),
                0, oNnz.empty() ? nullptr : oNnz.data()
            ),
            "MatMPIAIJSetPreallocation"
        );
        checkPetscError
        (
            MatSetOption(out, MAT_NEW_NONZERO_ALLOCATION_ERR, PETSC_FALSE),
```

Listing 2: Tripletes locales a matriz distribuida. Recortado. `[monodomainPetsc.H]` *(recortado; sigue en el fuente.)*

## 4. El offset global de fila

`gRowStart` es el desplazamiento de la primera fila de este rank, o sea `globalIndex::localStart()`. Es constante para toda la corrida — una malla, una descomposición — así que se fija una vez en `main()`, con `initialiseGlobalCells`, en vez de arrastrarlo por la firma de cada rutina. Vive en `monodomainBase.H` y es el mismo para los dos solvers.

Vale la pena detenerse acá porque es donde vivían cuatro bugs del port, todos del mismo tipo: escribir una columna *local* donde iba una *global*. En serie `gRowStart` es cero y `toGlobal()` es la identidad, así que **ninguno de los cuatro era visible sin correr en paralelo**. El test **M09** compara $K$ entrada por entrada contra el ensamblado serial deshaciendo la permutación con `cellProcAddressing`, y es lo único que puede verlos: un fingerprint de nnz y sumas de fila es invariante a mover una entrada de columna.

Justamente por eso el valor no se lee directo sino a través de `globalRowStart()`, que **se niega a servir el cero inicial**. La asignación ocurre en `main()`, después de leer la malla, y todas las inserciones de fila en PETSc la leen. Un camino de código que llegara a una matriz antes de ese punto usaría offset cero *en todos* los ranks, o sea que los ranks $> 0$ escribirían sus filas encima de las del rank 0: operador corrompido y respuesta plausible pero equivocada, nunca un error. Hoy nada lo hace; el accesor lo vuelve imposible en vez de meramente improbable.

```
    // Global row offset of this rank's block of cells, i.e.
    // globalIndex::localStart(). Constant for the whole run (one mesh, one
    // decomposition), so it is set once, by initialiseGlobalCells(), rather
    // than threaded through every linear-solver signature. Zero in serial.
    label gRowStart = 0;
    bool gRowStartSet = false;

    // Read through an accessor that refuses to serve the initial 0. Every
    // PETSc row insertion reads it; a code path that touched a matrix before
    // initialiseGlobalCells() would offset by 0 on EVERY rank, so ranks > 0
    // would write their rows on top of rank 0's: a corrupt operator and a
    // plausible wrong answer, never an error. This makes it impossible rather
    // than merely unlikely.
    label globalRowStart()
    {
        if (!gRowStartSet)
        {
            FatalErrorInFunction
                << "The global row offset was read before it was set."
                << " It is assigned once in main by initialiseGlobalCells();"
                << " reaching a PETSc matrix before that offsets every rank's"
                << " rows by 0 and corrupts the operator."
                << exit(FatalError);
        }

        return gRowStart;
    }
```

Listing 3: El offset, y el accesor que se niega a servirlo sin asignar. [`monodomainBase.H`]

```
    void initialiseGlobalCells(const fvMesh& mesh)
    {
        gCellsPtr.reset(new globalIndex(mesh.nCells()));
        gRowStart = gCellsPtr->localStart();
        gRowStartSet = true;
    }
```

Listing 4: Dónde se asigna, una sola vez. [`monodomainBase.H`]

## 5. Nombres de opciones: una sola ortografía, y los de PETSc

**Cada opción del diccionario tiene exactamente una ortografía**, comparada distinguiendo mayúsculas, y cualquier otra cosa detiene la corrida al arrancar con la lista de valores válidos. Lo hace `requireOneOf`, que se llama en `createFields.H` justo después de leer cada clave.

Reemplazó a una capa que pasaba todo a minúsculas y acumulaba alias (`sparselu`/`lu`, `picard`, `localDiagonal`, `RKT45`, `consistentHO`, `jakobi`…). No era cosmético: así se escondió una respuesta equivocada. Con `linearSolver SparseLU`, el camino de diagonalIion forzaba un solve directo, pero el solver cacheado de Picard recibía `preonly` con el PC por defecto, `ilu`: *una* aplicación de ILU haciéndose pasar por la inversa. Picard convergía al punto fijo del operador equivocado — $V_m$

con un error 4,6 veces mayor, cero pasos no convergidos, ninguna advertencia. `resolvePetscKspPc` resuelve ahora ese par en un solo lugar para los tres caminos PETSc.

Quedan dos vocabularios separados. `linearSolver` es neutral (`GMRES`, `BiCGSTAB`, `SparseLU`); las claves de PETSc usan los nombres propios de PETSc de listas cerradas. `petscKspTypeForLinearSolver` es lo único que traduce de uno a otro, para el valor por defecto de `petscLinearKspType`.

La función de esta sección que importa para interpretar resultados es `petscParallelPcTypeName`.

`ilu` y `lu` **no tienen implementación MPIAIJ**, así que en paralelo PETSc los envuelve en **block-Jacobi**, y el bloque *es* la partición. Es decir que a np=1 y a np=2 no se resuelve con el mismo precondicionador: son dos algoritmos distintos. `hypre` y `gamg` sí son paralelos nativos, pero su *coarsening* también depende de cómo quedó cortada la malla.

Sólo `jacobi` es el mismo operador a cualquier número de ranks, y ésa es la razón de que la suite lo fije en todos los tests de invariancia. El test **M07** lo cuantifica: la desviación entre np=1 y np=2 es **1.26e-11** con `jacobi` contra **3.60e-05** con `ilu`, un factor de 2.86 millones.

Para producción la respuesta es otra y está en **M18**: ahí lo que importa es el tiempo, y el ranking cambia con el paso de tiempo.

```
87      void requireOneOf
88      (
89          const char* key,
90          const word& value,
91          std::initializer_list<const char*> valid
92      )
93      {
94          for (const char* v : valid)
95          {
96              if (value == v)
97              {
98                  return;
99              }
100         }
101
102         std::string options;
103         for (const char* v : valid)
104         {
105             if (!options.empty())
106             {
107                 options += ", ";
108             }
109             options += v;
110         }
111
112         FatalErrorInFunction
113             << "Unknown " << key << ": " << value << nl
114             << "Valid options are " << options.c_str()
115             << " (exact spelling, case-sensitive)."
116             << exit(FatalError);
117     }
```

Listing 5: Una sola ortografía por opción, validada al arrancar. `[monodomainBase.H]`

```
344     void resolvePetscKspPc
345     (
346         const word& linearSolver,
347         const word& petscKspType,
348         const word& petscPcType,
349         word& kspType,
350         word& pcType
351     )
352     {
353         kspType = petscKspType;
354         pcType = petscPcType;
355
356         if (linearSolver == "SparseLU")
357         {
358             kspType = "preonly";
359             pcType = "lu";
```

```
        }
    }
```

Listing 6: El par (KSP, PC) de un linearSolver, en un solo lugar. [monodomainPetsc.H]

```
std::string petscParallelPcTypeName(const word& pcType, std::string& subPc)
{
    const std::string s(pcType);
    subPc.clear();

    if (Pstream::parRun() && (s == "ilu" || s == "lu"))
    {
        subPc = s;
        return "bjacobi";
    }

    return s;
}
```

Listing 7: La sustitución que hace que ilu deje de ser ilu en paralelo. [monodomainPetsc.H]

## 6. La solución manufacturada

Acá está lo que hace que este solver pueda demostrar un orden en vez de estimarlo. `exactVm` y las tres `exactU` dan el campo elegido; `computeF` y `computeG` son sus factores espaciales, escritos por dimensión.

`vmSourcePDE` es el término que se agrega a la ecuación para que ese campo sea solución exacta: se obtiene metiendo la solución elegida en el operador y despejando lo que falta.

`computeBeta` lee `conductivity[0]`, o sea la celda cero, y eso sólo es válido con conductividad **uniforme**: en una malla descompuesta la celda cero es una celda física distinta en cada rank. No es un item abierto sino un **supuesto asumido** — este solver existe para este caso manufacturado, donde $D$ es uniforme por diseño. Queda registrado acá porque es lo primero que habría que tocar si alguna vez entraran fibras, que hacen $D$ no uniforme por construcción.

**Hay dos soluciones manufacturadas**, y la clave `manufacturedSolution` elige. La de siempre, `sines`, es $\sqrt{1+t}\,\cos(\pi x)\cos(2\pi y)\cdots$: suave y no polinómica, así que ninguna reconstrucción de acá es exacta sobre ella y el orden *espacial* medido es el del esquema.

La otra, `uniform`, tiene todos los factores espaciales iguales a uno. Existe por una razón puntual: toda reconstrucción, cuadratura y flujo de este solver es exacto sobre un campo constante, así que el error espacial es cero a nivel de redondeo y el error contra la solución exacta es **puramente temporal**. Es la única forma de ajustar un orden temporal contra la solución analítica en vez de contra otra solución numérica, y es la vía B de **M02**. A malla fija con `sines` eso es imposible: el error espacial ($2{,}4\times10^{-5}$ en hexa $N=40$) entierra al temporal, que para BDF4 es $\sim 10^{-10}$.

Lo que cuesta, dicho para que nadie lea de más el resultado: el laplaciano de una constante es cero, así que bajo `uniform` la difusión no participa y lo que se verifica es la integración temporal del sistema de reacción. La difusión la cubre **M02** por autoconvergencia. Y `computeBeta` devuelve cero en ese modo, porque no hay laplaciano que cancelar.

```
scalar exactVm(const point& p, const scalar t, const label dim)
{
    return std::sqrt(1.0 + t)*computeF(p, dim);
}
```

Listing 8: El campo exacto. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
scalar computeF(const point& p, const label dim)
{
    // Exactly representable by every reconstruction here, so the spatial
    // error vanishes and what is left is the temporal error.
    if (mmsProfile == mmsUniform)
```

```
        {
            return 1.0;
        }

        const scalar pi = constant::mathematical::pi;

        if (dim == 1)
        {
            return std::cos(pi*p.x());
        }
        else if (dim == 2)
        {
            return std::cos(pi*p.x())*std::cos(2.0*pi*p.y());
        }

        return
            std::cos(pi*p.x())
           *std::cos(2.0*pi*p.y())
           *std::cos(3.0*pi*p.z());
    }
```

Listing 9: El factor espacial, y dónde el modo uniform lo vuelve 1. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
    scalar vmSourcePDE
    (
        const scalar V,
        const scalar u1,
        const scalar u2,
        const scalar u3,
        const scalar beta,
        const scalar chiVal,
        const scalar CmVal
    )
    {
        return -ionicCurrentPDE(V, u1, u2, u3, beta, chiVal, CmVal);
    }
```

Listing 10: El término que lo convierte en solución. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
    scalar computeBeta(const volTensorField& conductivity, const label dim)
    {
        // A constant has zero Laplacian, so there is nothing for beta to
        // cancel and the ionic current reduces to the term that supplies
        // dV/dt.
        if (mmsProfile == mmsUniform)
        {
            return 0.0;
        }

        const tensor& D = conductivity[0];
        const scalar pi2 = sqr(constant::mathematical::pi);

        if (dim == 1)
        {
            return -pi2*D.xx();
        }
        else if (dim == 2)
        {
            return -pi2*(D.xx() + 4.0*D.yy());
        }

        return -pi2*(D.xx() + 4.0*D.yy() + 9.0*D.zz());
    }
```

Listing 11: Lee la celda 0: sólo vale con D uniforme. [highOrderManufacturedFDAImplicitPETScDistributed.C]

## 7. Bloques de construcción del ensamblado

Antes de las dos matrices de rigidez vienen las piezas que las dos comparten.

`addTripletIfNeeded` es el punto por el que pasa *toda* entrada de la matriz. Descarta las que valen menos que `SMALL`, lo que hace que el patrón de esparsidad dependa del valor y, por lo tanto, de la partición al nivel del redondeo.

`addCellGradientDotCoeffs` agrega la fila de gradiente de una celda proyectada sobre un vector. Usa `globalCellStencils()`, y ahí vivió durante meses el que se creía el problema abierto más grande del proyecto: sobre las celdas pegadas a un corte de procesador el stencil **perdía un miembro** (17 en serie contra 16 en paralelo). Como el ajuste MLS se rehace sobre el conjunto reducido, no cambiaba sólo el término que faltaba — **refitaba toda la fila**. Eso era lo que hacía fallar a **M11** y, río abajo, a **M15**, y se había dado por un item de solids4foam que se mide y no se arregla.

**No lo era.** La causa estaba del lado del caso: la clave `Nn` del diccionario son las celdas que se AGREGAN sobre el mínimo que exige el grado, y los barridos la venían escribiendo como si fuera el TOTAL. La biblioteca sumaba el mínimo otra vez, así que los estencils medían $2\min + 10$ en vez de $\min + 10$ — 16/22/30 celdas en 2D en vez de 13/16/20, un 50 % más grandes de lo pretendido — y el halo no alcanzaba a cubrirlos. Con el tamaño correcto el stencil es invariante a la partición: **M11 mide 0 stencils distintos y M15 baja de 4.981e-06 a 6.953e-15**, los dos en PASS. La suite quedó 24/24.

`exchangeCoupledFaceRows` es la que permite que el término de estabilización cruce un corte: intercambia con el rank vecino las filas que hacen falta del otro lado. Es **colectiva**, y está deliberadamente fuera del lazo de chunks — adentro se trabaría en cuanto dos ranks tuvieran una cantidad distinta de chunks.

```
191     void addTripletIfNeeded
192     (
193         std::vector<Triplet>& triplets,
194         const label row,
195         const label col,
196         const scalar value
197     )
198     {
199         if (mag(value) > SMALL)
200         {
201             triplets.emplace_back(row, col, value);
202         }
203     }
```

Listing 12: Por acá pasa toda entrada de la matriz. [monodomainBase.H]

```
373     void addCellGradientDotCoeffs
374     (
375         std::vector<Triplet>& triplets,
376         const label row,
377         const scalar scale,
378         const label cellI,
379         const vector& d,
380         const highOrderInterp& LREInterp
381     )
382     {
383         if (mag(scale) <= SMALL)
384         {
385             return;
386         }
387 
388         const CompactListList<label>& stencils =
389             LREInterp.globalCellStencils();
390         const CompactListList<vector>& gradCoeffs = LREInterp.QRGradCoeffs();
391         // Modified for cardiacFoam: must be UList<label> BY VALUE, not
392                 // a labelList reference - see the note on
393                 // CompactListList in the header comment.
394                 const UList<label> curStencil = stencils[cellI];
395         const label selfCoeffI = curStencil.size();
```

```
396
397             // Added for cardiacFoam: the coefficient rows carry one
398             // trailing entry for the cell itself, so this index must
399             // be the last one. If the stencil silently came back
400             // empty, selfCoeffI is 0, which is still in range and
401             // reads a neighbour's coefficient instead.
402             if (selfCoeffI != gradCoeffs[cellI].size() - 1)
403             {
404                 FatalErrorInFunction
405                     << "Stencil/coefficient mismatch at cell " << cellI
406                     << ": stencil size " << selfCoeffI
407                     << ", coefficient row " << gradCoeffs[cellI].size()
408                     << " (expected row = size + 1)."
409                     << abort(FatalError);
410             }
411
412         forAll(curStencil, cI)
413         {
414             addTripletIfNeeded
415             (
416                 triplets,
417                 row,
418                 curStencil[cI],
419                 scale*(gradCoeffs[cellI][cI] & d)
420             );
421         }
422
423         addTripletIfNeeded
424         (
425             triplets,
426             row,
427             gCol(cellI),
428             scale*(gradCoeffs[cellI][selfCoeffI] & d)
429         );
430     }
```

Listing 13: La fila de gradiente. Recortada. [monodomainAssembly.H]

```
30      void exchangeCoupledFaceRows
31      (
32          const fvMesh& mesh,
33          const std::function
34          <
35              void(const label patchI, const label faceI, labelList&, scalarField&)
36          >& buildRow,
37          List<labelList>& nbrCols,
38          List<scalarField>& nbrCoeffs
39      )
40      {
41          nbrCols.setSize(mesh.nBoundaryFaces());
42          nbrCoeffs.setSize(mesh.nBoundaryFaces());
43
44          if (!Pstream::parRun())
45          {
46              return;
47          }
48
49          PstreamBuffers pBufs(Pstream::commsTypes::nonBlocking);
50
51          forAll(mesh.boundary(), patchI)
52          {
53              const fvPatch& patch = mesh.boundary()[patchI];
54              const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);
55
56              if (!ppPtr)
57              {
58                  continue;
59              }
60
61              List<labelList> sendCols(patch.size());
62              List<scalarField> sendCoeffs(patch.size());
63
64              forAll(patch, faceI)
65              {
66                  buildRow(patchI, faceI, sendCols[faceI], sendCoeffs[faceI]);
```

```
            }

            UOPstream toNbr(ppPtr->neighbProcNo(), pBufs);
            toNbr << sendCols << sendCoeffs;
        }

        pBufs.finishedSends();

        forAll(mesh.boundary(), patchI)
        {
            const fvPatch& patch = mesh.boundary()[patchI];
            const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);

            if (!ppPtr)
            {
                continue;
            }

            List<labelList> recvCols;
            List<scalarField> recvCoeffs;

            UIPstream fromNbr(ppPtr->neighbProcNo(), pBufs);
            fromNbr >> recvCols >> recvCoeffs;
```

Listing 14: Intercambio de filas a través de un corte. Recortada. `[monodomainAssembly.H]` *(recortado; sigue en el fuente.)*

## 8. Las dos matrices de rigidez

El solver tiene dos ensamblados del operador de difusión y la tríada del caso elige cuál corre.

`assembleStandardOrthogonalStiffnessMatrix` es el de bajo orden: flujo de dos puntos entre celdas vecinas. Usa el **promedio** $\frac{1}{2}(D_{own} + D_{nei})$ como tensor de cara.

`assembleHighOrderStiffnessMatrix` es el de alto orden, sobre el stencil MLS con cuadratura en las caras. Acá estuvo el defecto más grande que encontró la verificación: usaba `conductivity[own]` **en las dos filas** de una cara interna, en vez del promedio. Con $D$ uniforme los dos tensores coinciden y se cancela idénticamente, así que fue invisible durante toda la vida del código; con $D$ no uniforme hacía que $K$ **cambiara un 21 % al descomponer la malla**, porque al cortar, cada rank es dueño de su lado de la cara. Está corregido, y **M15** lo mide: de 2.108e-01 a 4.736e-15.

Las dos ensamblan **por chunks** de caras, y la clave del diccionario `assemblyFaceChunk` controla el tamaño. Es control de capacidad solamente: el resultado no debe depender de él, y **M12** lo verifica — con tolerancia y no bit a bit, porque cambiar el tamaño de bloque cambia el orden de suma en punto flotante.

```
    void assembleStandardOrthogonalStiffnessMatrix
    (
        const fvMesh& mesh,
        const volTensorField& conductivity,
        const highOrderInterp& LREInterp,
        const scalar stabilisationAlpha,
        SpMat& K
    )
    {
        const vectorField& C = mesh.C();
        const scalarField& V = mesh.V();
        const labelUList& owner = mesh.owner();
        const labelUList& neighbour = mesh.neighbour();

        std::vector<Triplet> triplets;
        triplets.reserve(4*mesh.nFaces());

        forAll(neighbour, faceI)
        {
            const label own = owner[faceI];
            const label nei = neighbour[faceI];
```

```
            const tensor Df = 0.5*(conductivity[own] + conductivity[nei]);
            const vector dPN = C[nei] - C[own];
            const scalar a = orthogonalDiffusionCoeff(mesh.Sf()[faceI], dPN, Df);

            addTripletIfNeeded(triplets, own, gCol(own), -a/max(V[own], SMALL));
            addTripletIfNeeded(triplets, own, gCol(nei),  a/max(V[own], SMALL));
            addTripletIfNeeded(triplets, nei, gCol(own),  a/max(V[nei], SMALL));
            addTripletIfNeeded(triplets, nei, gCol(nei), -a/max(V[nei], SMALL));

            // The tangential flux a two-point form would drop. grad V|_f is
            // taken as the average of the two cell gradients, which is what
            // OpenFOAM's correction does; here each of them is expanded into
            // the matrix through its reconstruction row, so the term is
            // IMPLICIT. Skipped where w is zero, which on an aligned orthogonal
            // mesh is every face.
            const vector w = nonOrthFluxVector(mesh.Sf()[faceI], dPN, Df);

            if (w != vector::zero)
            {
                const scalar areaHalf = 0.5*(mag(mesh.Sf()[faceI]) + VSMALL);

                for (const label cellI : {own, nei})
                {
                    addCellGradientDotCoeffs
                    (
                        triplets, own, areaHalf/max(V[own], SMALL), cellI, w,
                        LREInterp
                    );
                    addCellGradientDotCoeffs
                    (
                        triplets, nei, -areaHalf/max(V[nei], SMALL), cellI, w,
                        LREInterp
                    );
                }
            }
        }

        if (stabilisationAlpha > SMALL)
```

Listing 15: Bajo orden: promedio en la cara. Recortado. `[monodomainAssembly.H]` *(recortado; sigue en el fuente.)*

```
    void assembleHighOrderStiffnessMatrix
    (
        const fvMesh& mesh,
        const volTensorField& conductivity,
        const highOrderInterp& LREInterp,
        const scalar stabilisationAlpha,
        const label tripletsPerFaceReserve,
        const label faceChunk,
        SpMat& K
    )
    {
        const bool twoD = mesh.nGeometricD() == 2;
        const vectorField& C = mesh.C();
        const surfaceVectorField& Cf = mesh.Cf();
        const CompactListList<point>& faceQP = LREInterp.faceQuadPoints();
        const CompactListList<scalar>& faceQW = LREInterp.faceQuadWeightPhysical();
        const CompactListList<label>& faceStencils =
            LREInterp.globalFaceStencils();
        const List<CompactListList<vector>>& faceGradCoeffs =
            LREInterp.QRGradFaceGPCoeffs();

        const scalarField& V = mesh.V();
        const labelUList& owner = mesh.owner();
        const labelUList& neighbour = mesh.neighbour();

        const label nCells = mesh.nCells();
        const label nInternalFaces = neighbour.size();

        std::vector<Triplet> triplets;
        const label reserveFaces = min(max(mesh.nFaces(), label(1)), faceChunk);
        triplets.reserve(reserveFaces*tripletsPerFaceReserve);

```

```
1432        SpMat Klocal(nCells, gNCols());
1433        bool kInitialised = false;
1434
1435        auto flushChunk =
1436        [&]()
1437        {
1438            if (triplets.empty())
1439            {
1440                return;
1441            }
1442
1443            Klocal.setZero();
1444            Klocal.setFromTriplets(triplets.begin(), triplets.end());
1445            Klocal.makeCompressed();
1446
1447            if (!kInitialised)
1448            {
1449                K.swap(Klocal);
1450                Klocal.resize(nCells, gNCols());
1451                kInitialised = true;
1452            }
1453            else
1454            {
1455                K += Klocal;
1456            }
1457
1458            triplets.clear();
1459        };
```

Listing 16: Alto orden, sobre el stencil MLS. Recortado. `[monodomainAssembly.H]` *(recortado; sigue en el fuente.)*

## 9. Resolución del sistema lineal

`solveSparseSystem` despacha entre PETSc y Eigen según `linearSolverBackend`. En paralelo sólo el camino de PETSc sirve, y ahora eso **se hace cumplir** en vez de sólo estar comentado: la matriz es `nLocal` × `nGlobal`, o sea rectangular, y las factorizaciones de Eigen necesitan una matriz cuadrada. Sin el guard, `SparseLU` moría en un `eigen_assert` que nombra un archivo interno de la librería — ni este solver, ni la clave del diccionario que lo eligió, ni la palabra "paralelo". Medido a np=2: el error ahora dice `200 local rows and 400 global columns`.

Hay además un GMRES escrito a mano (`solveGMRES`) que queda como camino de respaldo cuando no se usa PETSc. Ése tiene su propio guard, y por un motivo **distinto**: no hay matriz que pueda ser rectangular, pero sus productos internos (`V[i].dot(w)`, `w.norm()`) los calcula Eigen sobre el vector *local* y nunca se reducen. Cada rank armaría su propia base de Krylov y su propio test de convergencia, mientras el producto matriz-vector que llama *sí* es colectivo (`MatMult` de PETSc). Dos ranks que salgan del lazo en distinta iteración se traban ahí: no es un abort, es un cuelgue.

Un tercer punto de la misma familia: `usesPetscBackend` **valida** el nombre en vez de tratarlo como una lista blanca de dos entradas cuyo `else` era Eigen. Cualquier valor que no fuera `petsc`/`ksp` — un typo en un diccionario, por ejemplo — mudaba la corrida entera al backend serial sin una línea en el log. Es la misma falla que `nonlinearMethod Picrad` en el solver electro, que colapsaba una matriz de métodos sobre un solo método.

**Qué solver de Krylov corresponde** lo decide la simetría del operador, y está medida en **M10**: el de bajo orden es simétrico a 8.4e-16, el de alto orden **no** — 7.9 % de asimetría relativa, con 300 de 4292 entradas sin transpuesta. Con la tríada de alto orden hay que usar `gmres`. `cg` no falla siempre ahí, que es lo que lo hace peligroso: sobrevive mientras le alcancen pocas iteraciones y se cae cuando necesita más (**E13** lo muestra fallando con `jacobi` y `gamg`).

**El KSP para sobre el residuo verdadero (2026-10).** GMRES y BiCGStab precondicionan por izquierda por defecto, y su tolerancia se aplica al residuo *precondicionado*. Con `ilu` a $\Delta t$ grande los

dos diferían en $\sim 4\times$: el solve lineal "convergía" a $10^{-12}$ dejando un residuo verdadero de $3{,}66\times10^{-12}$, y Picard repetía ese mismo solve hasta el tope de iteraciones — 2000 por paso. Ahora el KSP cacheado usa precondicionamiento por **derecha** (como ya hacía el KSP shell de JFNK), cuya norma monitoreada es la verdadera; `cg` no tiene lado derecho y monitorea la norma sin precondicionar. Va antes de `KSPSetFromOptions`, así que una opción de PETSc todavía puede cambiarlo.

```
    bool usesPetscBackend(const word& backend)
    {
        requireOneOf("linear solver backend", backend, {"PETSc", "Eigen"});
        return backend == "PETSc";
    }
```

Listing 17: El nombre del backend, validado. [monodomainPetsc.H]

```
    EigVec solveSparseSystem
    (
        const SpMat& A,
        const EigVec& b,
        const word& linearSolverBackend,
        const word& linearSolver,
        const word& petscKspType,
        const word& petscPcType,
        const label petscRestart,
        const word& petscOptionsPrefix,
        const bool petscUseOptions,
        const scalar tol,
        const label maxIter,
        label& linearIterations,
        scalar& linearError,
        PetscKspMatrixSolver& petscSolver,
        PersistentEigenSolvers& eigenSolvers,
        const MatrixUpdate updatePolicy,
        const bool reusePreconditioner
    )
    {
        if (usesPetscBackend(linearSolverBackend))
        {
            // Reuse the persistent KSP/factorisation across time steps and
            // nonlinear iterations (Phase B). The matrix is only reassembled
            // into PETSc when the pattern changes (Rebuild) or its values
            // change (ValuesOnly); a constant operator (Reuse) skips both.
            switch (updatePolicy)
            {
                case MatrixUpdate::Rebuild:
                {
                    word kspType;
                    word pcType;
                    resolvePetscKspPc
                    (
                        linearSolver, petscKspType, petscPcType, kspType, pcType
                    );

                    petscSolver.reset
                    (
                        A,
                        kspType,
                        pcType,
                        tol,
                        maxIter,
                        petscRestart,
                        petscOptionsPrefix,
                        petscUseOptions
                    );
                    break;
                }

                case MatrixUpdate::ValuesOnly:
                {
                    petscSolver.updateValues(A, reusePreconditioner);
                    break;
                }

                case MatrixUpdate::Reuse:
```

```
731                 {
```

Listing 18: El despacho entre backends, con el guard de paralelo. Recortado. [monodomainKrylov.H] *(recortado; sigue en el fuente.)*

```
569     EigVec solveSparseSystemEigen
570     (
571         const SpMat& A,
572         const EigVec& b,
573         const word& linearSolver,
574         const scalar tol,
575         const label maxIter,
576         label& linearIterations,
577         scalar& linearError,
578         PersistentEigenSolvers& persistent,
579         const MatrixUpdate updatePolicy
580     )
581     {
582         if (linearSolver == "SparseLU")
583         {
584             Eigen::SparseLU<SpMat>& solver = persistent.lu;
585 
586             // Symbolic analysis depends only on the sparsity pattern, which is
587             // constant for the whole run: do it once. The numeric factorisation
588             // is redone whenever the values change (Rebuild / ValuesOnly) and
589             // skipped entirely when the matrix is unchanged (Reuse).
590             if (updatePolicy != MatrixUpdate::Reuse)
591             {
592                 if (!persistent.luAnalyzed)
593                 {
594                     solver.analyzePattern(A);
595                     persistent.luAnalyzed = true;
596                 }
597 
598                 solver.factorize(A);
599 
600                 if (solver.info() != Eigen::Success)
601                 {
602                     FatalErrorInFunction
603                         << "SparseLU factorization failed"
604                         << exit(FatalError);
605                 }
606             }
607 
608             EigVec x = solver.solve(b);
609             linearIterations = 1;
610             linearError = 0.0;
611 
612             if (solver.info() != Eigen::Success)
613             {
614                 FatalErrorInFunction
615                     << "SparseLU solve failed"
616                     << exit(FatalError);
617             }
618 
619             return x;
620         }
621         else if (linearSolver == "BiCGSTAB")
622         {
623             Eigen::BiCGSTAB<SpMat, Eigen::IncompleteLUT<scalar>>& solver =
624                 persistent.bicgstab;
625             solver.setTolerance(tol);
626             solver.setMaxIterations(maxIter);
627 
628             // compute() builds the incomplete-LU preconditioner; reuse it when
```

Listing 19: El camino serial. Recortado. [monodomainKrylov.H] *(recortado; sigue en el fuente.)*

```
655             // Stop on the TRUE residual, the one the nonlinear stopping test
656             // measures. GMRES and BiCGStab default to left preconditioning,
657             // whose tolerance applies to the preconditioned residual: with ilu
658             // at a large dt the two differed by ~4x, the linear solve
659             // "converged" to 1e-12 and left a true residual of 3.66e-12, and
660             // Picard repeated that solve up to the iteration cap. Right
661             // preconditioning, as the JFNK shell solver already uses; cg has no
```

```
662             // right side, so it monitors the unpreconditioned norm instead.
663             // Before KSPSetFromOptions, so PETSc options can still override.
664             if (isGmresType(kspType) || kspName == "bcgs")
665             {
666                 checkPetscError(KSPSetPCSide(ksp_, PC_RIGHT), "KSPSetPCSide");
667             }
668             else if (kspName == "cg")
669             {
670                 checkPetscError
671                 (
672                     KSPSetNormType(ksp_, KSP_NORM_UNPRECONDITIONED),
673                     "KSPSetNormType"
674                 );
675             }
```

Listing 20: Precondicionamiento por derecha en el KSP cacheado. `[monodomainPetsc.H]`

## 10. Discretización temporal: las dos familias

El solver ofrece dos familias de integrador temporal, y `implicitScheme` elige entre ellas. Con el orden que cada una debe entregar:

| `implicitScheme` | orden | estabilidad |
|---|---|---|
| `backwardEuler` | $O(\Delta t)$ | A y L-estable |
| `crankNicolson` | $O(\Delta t^2)$ | A-estable, **no** L-estable |
| `BDF1` | idéntico a `backwardEuler`, bit a bit | A y L |
| `BDF2` | $O(\Delta t^2)$ | A y L-estable |
| `BDF3` | $O(\Delta t^3)$ | $A(86{,}03°)$ |
| `BDF4` | $O(\Delta t^4)$ | $A(73{,}35°)$ |

`timeSchemeCoeffs` traduce el nombre a los coeficientes. Lo que hay que ver en esa función es que **las dos familias terminan en el mismo par de operadores**: $A = (a_0/\Delta t)M - \theta L$ y $B = M/\Delta t + (1-\theta)L$. BDF cambia *sólo* el escalar que multiplica a $M$, así que el ensamblado, el patrón de esparsidad y la factorización cacheada quedan intactos. Y el historial entra por un *único* producto por la matriz de masa, porque la combinación $H^n = -\sum_{j\geq 1} a_j V^{n+1-j}$ se arma como campo antes de aplicar $B$: un paso BDF-$k$ cuesta un producto matriz–vector, no $k$.

BDF5 y BDF6 no se ofrecen. Pasada la segunda barrera de Dahlquist ningún multipaso es A-estable, y la cuña se cierra de 86° y 73° en BDF3/4 a 51,8° y 17,8° en BDF5/6.

**El arranque es la parte que engaña.** Un método de $k$ pasos necesita valores iniciales precisos a $O(\Delta t^k)$ *respecto del sistema semi-discreto*, no de la PDE. Sembrar con la solución manufacturada da $O(\tau_h\,\Delta t)$ — primer orden — porque la solución exacta de la PDE no satisface las ecuaciones semi-discretas. Nada avisa: la corrida termina bien y sólo el orden sale mal, que es por lo que `bdfStartup` tiene hoy `ESDIRK3` **por defecto**.

Con ese arranque los órdenes son los nominales. Medido a $N = 40$ por las dos vías que usa la suite — **M02**, autoconvergencia de Cauchy sobre la solución de senos con la difusión activa, y su vía B, error contra la solución analítica sobre la MMS uniforme:

| `implicitScheme` | nominal | Cauchy, vía A | analítica, vía B |
|---|---|---|---|
| `backwardEuler` y `BDF1` | 1 | 1,000 | 1,000 |
| `crankNicolson` | 2 | 1,998 | 2,000 |
| `BDF2` | 2 | 1,979 | 1,992 |
| `BDF3` | 3 | 2,932 | 2,971 |
| `BDF4` | 4 | 3,891 | 3,908 |

`exact` y `constant` se conservan, pero **sólo** para que el costo de un mal arranque siga siendo medible: es lo que hace **M20**, aislando el efecto como $e(\Delta t) = \|V_{\text{arranque}} - V_{\text{ESDIRK3}}\|$ al mismo $\Delta t$ y la misma malla, donde el error propio de BDF3 se cancela. Los dos modos resultan $O(\Delta t)$, y difieren donde la teoría dice: el de `constant` no cambia al refinar la malla (razón 1,00) y el de `exact` cae 15,5 veces por refinamiento, porque su coeficiente es $\tau_h$.

Por eso existe `esdirk3Tableau`, y **sólo** por eso: ESDIRK no es seleccionable como esquema. Es autoarrancable, que es lo que el arranque pide, pero tiene orden de etapa 2 y sufre reducción de orden — ESDIRK4 midió 3,03 donde BDF4 llega a 3,89; esa cifra de ESDIRK4 es anterior al arreglo del lazo temporal y no se volvió a medir — y cada etapa reintegra las EDO, que es el 74–99 % del tiempo en el solver fisiológico.

```
    TimeSchemeCoeffs timeSchemeCoeffs(const word& scheme)
    {
        TimeSchemeCoeffs c;

        if (scheme == "backwardEuler")
        {
            c.theta = 1.0;
            c.bdfOrder = 0;
            return c;
        }

        if (scheme == "crankNicolson")
        {
            c.theta = 0.5;
            c.bdfOrder = 0;
            return c;
        }

        // BDF is fully implicit in K, so theta = 1, and the source is wanted at
        // t^{n+1} only - one source evaluation per step FEWER than
        // Crank-Nicolson needs.
        c.theta = 1.0;

        if (scheme == "BDF1")
        {
            c.bdfOrder = 1;
            c.a[0] =  1.0;
            c.a[1] = -1.0;
        }
        else if (scheme == "BDF2")
        {
            c.bdfOrder = 2;
            c.a[0] =  3.0/2.0;
            c.a[1] = -2.0;
            c.a[2] =  1.0/2.0;
        }
        else if (scheme == "BDF3")
        {
            c.bdfOrder = 3;
            c.a[0] =  11.0/6.0;
            c.a[1] = -3.0;
            c.a[2] =  3.0/2.0;
            c.a[3] = -1.0/3.0;
        }
        else if (scheme == "BDF4")
        {
            c.bdfOrder = 4;
            c.a[0] =  25.0/12.0;
            c.a[1] = -4.0;
            c.a[2] =  3.0;
            c.a[3] = -4.0/3.0;
            c.a[4] =  1.0/4.0;
        }
        else
        {
            FatalErrorInFunction
                << "Unknown implicitScheme: " << scheme << nl
                << "Valid options are backwardEuler, crankNicolson, BDF1,"
                << " BDF2, BDF3 and BDF4." << nl
```

```
                << "ESDIRK is not a scheme here: it exists only to supply the"
```

Listing 21: Los coeficientes de las dos familias. Recortado. [monodomainTimeSchemes.H] *(recortado; sigue en el fuente.)*

```
    EsdirkTableau esdirk3Tableau()
    {
        EsdirkTableau tb;
        tb.nStages = 4;
        tb.order = 3;
        tb.gamma = 0.435866521508458999416;

        forAll(tb.c, i)
        {
            tb.c[i] = 0.0;
            forAll(tb.a[i], j)
            {
                tb.a[i][j] = 0.0;
            }
        }

        const scalar g = tb.gamma;

        tb.c[0] = 0.0;
        tb.c[1] = 2.0*g;
        tb.c[2] = 0.6;
        tb.c[3] = 1.0;

        tb.a[1][0] = g;
        tb.a[1][1] = g;

        tb.a[2][0] =  0.257648246066427245800;
        tb.a[2][1] = -0.093514767574886245216;
        tb.a[2][2] =  g;

        tb.a[3][0] =  0.187641024346723825161;
        tb.a[3][1] = -0.595297473576954948048;
        tb.a[3][2] =  0.971789927721772123471;
        tb.a[3][3] =  g;

        return tb;
    }
```

Listing 22: La tabla del arranque, derivada de las condiciones de orden. [monodomainTimeSchemes.H]

```
        // The ESDIRK startup step. This runs ONLY for
        // the first bdfOrder-1 steps of a BDF run, to build the history; it is
        // not reachable as a scheme in its own right (see the note on
        // needEsdirkOperator above for why not).
        //
        // Stage 1 is explicit, Y_1 = V^n. For i >= 2 the stage solves
        //
        //   A_g Y_i = (1/g) [ B V^n + sum_{j<i} a_ij F_j ] + bc(t_i) + s(Y_i)
        //
        // with A_g = M/(g dt) - lapScale K, B = M/dt, t_i = t + c_i dt, and
        //
        //   F_j = lapScale K Y_j + bc(t_j) + s(Y_j).
        //
        // Note bc enters with coefficient 1 while the B and F terms carry 1/g:
        // that falls out of dividing the stage equation by g dt. Getting it
        // wrong would be invisible on this MMS suite, where every case is
        // zeroGradient and bc is identically zero - hence this note.
        //
        // The shape is the same one-step equation the Picard branch below
        // solves, which is why one matrix, one cached factorisation and one
        // fixed-point iteration serve every stage.
        //
        // Stiffly accurate (c_s = 1, b = last row): V^{n+1} = Y_s, the last
        // stage IS the answer and there is no separate combination step.
        //
        // Only Picard is wired up. JFNK and diagonalIon would each need their
        // per-stage Jacobian machinery threaded through, which is separate
        // work; an unknown scheme name is a FatalError upstream, so nothing
        // reaches here by accident.
        // ESDIRK runs only while it is supplying the first bdfOrder-1 starting
```

```
3829            // values. After that the history is full and BDF takes over for the
3830            // rest of the run - which, for anything but the opening steps, is the
3831            // whole run.
3832            // Modified for cardiacFoam: counted in VALID levels rather than in
3833            // steps, so that a rolled-back step can restart the history (see the
3834            // rotation at the end of the step), as the physiological solver does.
3835            const bool esdirkThisStep =
3836                (bdfOrder > 1 && esdirkStartup && bdfValidLevels < bdfOrder - 1);
```

Listing 23: Dónde el arranque ESDIRK es alcanzable, y sólo ahí. [highOrderManufacturedFDAImplicitPETScDistributed.C]

## 11. El driver de Vm que manejan las EDO

Las EDO de estado se integran con sub-pasos *dentro* del paso de la PDE, y en cada sub-paso necesitan un $V_m$ en un instante intermedio que el esquema no conoce. Cómo se rellena ese hueco **acota el orden de todo el solver**: una recta entre los dos extremos es $O(\Delta t^2)$, y ningún integrador temporal por encima de segundo orden puede mostrar el suyo detrás de eso.

No es una salvedad teórica, está medido. La clave `stateODEDriver` conmuta entre las dos opciones — `history`, el interpolante por los niveles que el esquema ya guarda, y `linear`, la rampa vieja — y **M21** corre BDF4 con cada una sin cambiar nada más: la rampa ajusta **2,04** contra **3,89** del interpolante, con el error del par más fino mayor por un factor 44,7. El 2,04 es exactamente lo que predice la cuenta: la rampa le erra por $O(\Delta t^2)$ punto a punto, que integrado sobre el paso es $O(\Delta t^3)$ local y $O(\Delta t^2)$ global.

`history` es el valor por defecto; `linear` existe para que ese costo siga siendo medible, porque un orden capado es todo lo que la corrida reporta — no hay error ni advertencia.

**El camino de puntos de Gauss (2026-10).** Con `gaussPointODE` la EDO se integra en cada punto de cuadratura, y hasta octubre lo hacía siempre con la rampa. La clave `stateODEGaussPointHistory` (por defecto `false` en el solver, `true` en el harness de la suite) le da el mismo polinomio que al camino por celda *sin guardar nada entre pasos*: al principio de cada paso se reconstruyen en los puntos de Gauss los niveles de la historia BDF (o los valores de etapa ESDIRK), con el mismo operador que da $V^n$ ahí (`reconstructVmLevelAtIionIntegrationPoints`). Son $k-1$ arreglos temporales del tamaño de los puntos de Gauss, sólo dentro del paso.

Medido en **M02** con GP, a $N = 40$: con la clave, BDF3 da 2,932 (vía A) y 2,971 (vía B), BDF4 da 3,891 y 3,908 — los mismos órdenes que el camino por celda. Sin ella, por la vía B BDF3 da 2,90 y BDF4 se estanca en un error de $\sim 2 \times 10^{-10}$ (pares de 3,89, 7,51, −1,59, 1,63). El costo, medido en el electro (**E36**), es +9 % de tiempo en GP con BDF4.

**Un error de paso en las etapas ESDIRK, corregido.** En el mismo camino de Gauss, `updateIntegrationPoin` recibía `dt` en vez de `stageDt`: dentro de una etapa ESDIRK los estados se integraban sobre el paso entero, no sobre $c_i\,\Delta t$ como en el camino por celda. Sólo afectaba al arranque de los BDF en GP, y por eso no se había visto.

`interpolateVmDriver` evalúa el polinomio de Lagrange a través de los voltajes que el esquema exterior ya calculó: dos nodos para las familias $\theta$ y BDF1, $k+1$ niveles de historial para BDF-$k$, y los valores de etapa para el arranque ESDIRK. No cuesta ninguna evaluación extra — los nodos ya existen. El caso de dos nodos está escrito aparte y no pasa por el bucle, que es lo que garantiza que `backwardEuler` y `crankNicolson` sigan dando resultados idénticos bit a bit.

```
217     inline scalar interpolateVmDriver
218     (
219         const VmDriverNodes& nd,
220         const scalar s
221     )
222     {
223         // Written out for two nodes: the path every one-step scheme takes.
```

```
        if (nd.nNodes <= 2)
        {
            return (1.0 - s)*nd.v[0] + s*nd.v[1];
        }

        scalar result = 0.0;

        for (label j = 0; j < nd.nNodes; ++j)
        {
            scalar Lj = 1.0;

            for (label m = 0; m < nd.nNodes; ++m)
            {
                if (m != j)
                {
                    Lj *= (s - nd.s[m])/(nd.s[j] - nd.s[m]);
                }
            }

            result += Lj*nd.v[j];
        }

        return result;
    }
```

Listing 24: El interpolante, con el caso de dos nodos escrito aparte. [monodomainTimeSchemes.H]

```
    void reactionRatesLinearVm
    (
        const VmDriverNodes& vmNodes,
        const scalar tau,
        const scalar dt,
        const scalar u1,
        const scalar u2,
        const scalar u3,
        scalar& du1dt,
        scalar& du2dt,
        scalar& du3dt
    )
    {
        const scalar alpha =
            dt > SMALL
          ? min(max(tau/dt, scalar(0.0)), scalar(1.0))
          : scalar(1.0);

        const scalar VmTau = interpolateVmDriver(vmNodes, alpha);

        reactionRates(VmTau, u1, u2, u3, du1dt, du2dt, du3dt);
    }
```

Listing 25: Dónde el driver entra a las EDO. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
        // The same driver at the Gauss points, for the gaussPointODE path: the
        // linear blend from V^n by default, and under stateODEGaussPointHistory
        // the history polynomial, set below with its levels reconstructed at
        // the Gauss points (as in the physiological solver).
        VmDriverStages ipDriver;
        ipDriver.nNodes = 2;
        ipDriver.s[0] = 0.0;
        ipDriver.s[1] = 1.0;
        ipDriver.field[0] = &VmOldIntegrationPoints;
        const bool gaussHistory =
            stateODEGaussPointHistory && historyStateODEDriver
         && useHighOrder_Iion && dim > 1 && !reconstructStatesFromCellCentres;
        autoPtr<volScalarField> VmLevelScratchPtr;
        List<scalarField> VmBdfOlderIntegrationPoints;
        if (gaussHistory)
        {
            VmLevelScratchPtr.reset(new volScalarField("VmLevelScratch", VmOld));
        }
```

Listing 26: El driver de los puntos de Gauss y cuándo usa la historia. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
                    // Modified for cardiacFoam: stageDt, not dt. Inside an
                    // ESDIRK stage the states are integrated over c_i dt, as
                    // on the cell path; passing dt integrated them over the
                    // whole step at every stage.
                    updateIntegrationPointStatesODE
                    (
                        ipDriver,
                        VmCandidateIntegrationPoints,
                        u1OldIntegrationPoints,
                        u2OldIntegrationPoints,
                        u3OldIntegrationPoints,
                        u1CandidateIntegrationPoints,
                        u2CandidateIntegrationPoints,
                        u3CandidateIntegrationPoints,
                        stageDt,
                        stateODESolver,
                        stateODEInitialStep,
                        stateODEAbsTol,
                        stateODERelTol,
                        stateODEMaxSteps,
                        stateODEUseOpenMP,
                        stateODEOpenMPThreshold
```

Listing 27: El camino de Gauss con stageDt. [highOrderManufacturedFDAImplicitPETScDistributed.C]

## 12. Reconstrucción a los puntos de integración

Con alto orden, $V_m$ y los estados viven en los centros de celda pero la corriente iónica se evalúa en los puntos de Gauss. Estas rutinas hacen la reconstrucción MLS de ida y el promedio de vuelta.

Las reconstrucciones viven en `monodomainReconstruction.H`, compartidas con el electro. `reconstructCellFields` reconstruye varios campos en una sola pasada (los tres estados del MMS a la vez: hacerlo campo por campo repetía el recorrido y costaba un 30 % más en M18), con un solo intercambio de halo por campo fuera de la región de hilos.

Esa rutina acepta un limitador al rango de los vecinos (tipo Barth–Jespersen), pero **el MMS nunca lo pide**: sobre un campo suave sólo recortaría extremos legítimos. Lo usa, opcionalmente, el solver electro.

```
    void reconstructVmAtIonIntegrationPoints
    (
        const volScalarField& Vm,
        const Switch useHighOrderVm,
        const highOrderInterp& LREInterp_Vm,
        const highOrderInterp& LREInterp_Iion,
        scalarField& VmIntegrationPoints,
        const Switch useOpenMP,
        const label openMPThreshold,
        const Switch profileTimings,
        FineGrainedTimings& fineTimings
    )
    {
        const fvMesh& mesh = Vm.mesh();
        const CompactListList<point>& cellIonQuadP =
            LREInterp_Iion.cellQuadPoints();

        if (useHighOrderVm)
        {
            auto write = [&](const label ip, const label, const scalar v)
            {
                VmIntegrationPoints[ip] = v;
            };

            const scalarField* field = &Vm.primitiveField();

            reconstructCellFieldsAtIntegrationPoints
            (
                mesh,
```

```
                UList<const scalarField*>(&field, 1),
                LREInterp_Vm,
                cellIionQuadP,
                write,
                nullptr,
                false,
                useOpenMP,
                openMPThreshold,
                profileTimings ? &fineTimings.vmReconDerivWallTime : nullptr,
                profileTimings ? &fineTimings.vmReconExpandWallTime : nullptr
            );

            if (profileTimings)
            {
                ++fineTimings.vmReconCalls;
            }
            return;
        }

        // Low order: fvc::grad is a library call; the expansion is threaded.
        const vectorField& C = mesh.C();
        const label nCells = mesh.nCells();
        const labelList& ipOffsets = cellIionQuadP.offsets();
        const scalarField& VmI = Vm.primitiveField();

        tmp<volVectorField> tGradVm = fvc::grad(Vm);
        const vectorField& gradVm = tGradVm->internalField();

        #pragma omp parallel for schedule(static) \
                if(useOpenMP && nCells >= openMPThreshold)
        for (label cellI = 0; cellI < nCells; ++cellI)
```

Listing 28: Reconstrucción de Vm. Recortada. `[monodomainReconstruction.H]` *(recortado; sigue en el fuente.)*

```
    void reconstructStatesAtIionIntegrationPoints
    (
        const volScalarField& u1,
        const volScalarField& u2,
        const volScalarField& u3,
        const highOrderInterp& LREInterp_states,
        const highOrderInterp& LREInterp_Iion,
        scalarField& u1IntegrationPoints,
        scalarField& u2IntegrationPoints,
        scalarField& u3IntegrationPoints,
        const Switch useOpenMP,
        const label openMPThreshold,
        const Switch profileTimings,
        FineGrainedTimings& fineTimings
    )
    {
        const fvMesh& mesh = u1.mesh();
        const CompactListList<point>& qp = LREInterp_Iion.cellQuadPoints();

        // The three states in one walk of the stencil, as this solver's own
        // loop did before the shared core existed.
        const scalarField* fieldsArr[3] =
            {&u1.primitiveField(), &u2.primitiveField(), &u3.primitiveField()};
        scalarField* outputs[3] =
            {&u1IntegrationPoints, &u2IntegrationPoints, &u3IntegrationPoints};

        auto write = [&outputs](const label ip, const label k, const scalar v)
        {
            (*outputs[k])[ip] = v;
        };

        reconstructCellFieldsAtIntegrationPoints
        (
            mesh,
            UList<const scalarField*>(fieldsArr, 3),
            LREInterp_states,
            qp,
            write,
            nullptr,
            false,
```

```
            useOpenMP,
            openMPThreshold,
            profileTimings ? &fineTimings.stateReconDerivWallTime : nullptr,
            profileTimings ? &fineTimings.stateReconExpandWallTime : nullptr
        );

        if (profileTimings)
        {
            ++fineTimings.stateReconCalls;
        }
    }
```

Listing 29: Los estados, sin limitador. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
    void averageIntegrationPointFieldToCells
    (
        const scalarField& integrationPointValues,
        const highOrderInterp& LREInterp_Iion,
        volScalarField& field
    )
    {
        const fvMesh& mesh = field.mesh();
        const CompactListList<scalar>& cellIionQuadW =
            LREInterp_Iion.cellQuadWeightPhysical();

        scalarField& cellValues = field.primitiveFieldRef();
        label integrationPointI = 0;

        forAll(mesh.cells(), cellI)
        {
            scalar valueBar = 0.0;
            scalar wSum = 0.0;

            forAll(cellIionQuadW[cellI], qI)
            {
                const scalar w = cellIionQuadW[cellI][qI];
                valueBar += w*integrationPointValues[integrationPointI];
                wSum += w;
                ++integrationPointI;
            }

            cellValues[cellI] = valueBar/max(wSum, SMALL);
        }

        field.correctBoundaryConditions();
    }
```

Listing 30: El promedio de vuelta a celdas. [highOrderManufacturedFDAImplicitPETScDistributed.C]

## 13. Integración de las EDO, y el paralelismo por hilos

`advanceStateODE` avanza los estados en cada celda o punto de integración, con RK4 o RKF45 adaptativo según el diccionario.

Acá están los lazos con OpenMP del `.C`. Hay otros dos lugares con hilos, los dos en el código común: las reconstrucciones a los puntos de Gauss (`monodomainReconstruction.H`) y los productos matriz–vector de Eigen, que `openMPControls.H` ata a la misma clave con `Eigen::setNbThreads` (con `stateODEUseOpenMP false` todo es serial). **El ensamblado no está paralelizado con hilos**, sólo por MPI. Las regiones funcionan porque cada punto integra su propia EDO, o reconstruye su propio valor, sin hablar con ningún otro, y por eso el resultado es **bit-idéntico** con cualquier cantidad de hilos, cosa que **E20** mide en el otro solver dando exactamente cero.

El umbral por debajo del cual la región no se abre llega como parámetro desde la clave `stateODEOpenMPThreshold`, en todas las llamadas que lo pasan. Esto figuró un tiempo como item abierto — se creía que el valor estaba fijo en 256 en seis lazos y que la clave no hacía nada — y al ir a arreglarlo resultó que ya

estaba conectado y que lo desactualizado era el registro. Lo que sí se hizo fue **quitar los argumentos por defecto** de las seis firmas que reciben el umbral, para que una llamada nueva que se olvide de pasarlo no compile, en vez de correr en silencio con un 256 escrito en la firma.

```
    void advanceStateODE
    (
        const VmDriverNodes& vmNodes,
        const scalar u1Old,
        const scalar u2Old,
        const scalar u3Old,
        const scalar dt,
        const word& stateODESolver,
        const scalar stateODEInitialStep,
        const scalar stateODEAbsTol,
        const scalar stateODERelTol,
        const label stateODEMaxSteps,
        scalar& u1New,
        scalar& u2New,
        scalar& u3New
    )
    {
        if (dt <= SMALL)
        {
            u1New = u1Old;
            u2New = u2Old;
            u3New = u3Old;
            return;
        }

        if (stateODESolver == "Euler")
        {
            scalar du1 = 0.0, du2 = 0.0, du3 = 0.0;
            reactionRatesLinearVm
            (
                vmNodes, 0.0, dt,
                u1Old, u2Old, u3Old,
                du1, du2, du3
            );

            u1New = u1Old + dt*du1;
            u2New = u2Old + dt*du2;
            u3New = u3Old + dt*du3;
            return;
        }

        if (stateODESolver == "RK4")
        {
            rk4StateStep
            (
                vmNodes, 0.0, dt, dt,
                u1Old, u2Old, u3Old,
                u1New, u2New, u3New
            );
            return;
        }

        // Validated at start-up in createFields.H, so this is a guard, not a
        // reachable branch.
        if (stateODESolver != "RKF45")
        {
            FatalErrorInFunction
                << "Unknown state ODE solver: " << stateODESolver << nl
                << "Valid options are RKF45, RK4 and Euler"
                << exit(FatalError);
```

Listing 31: El avance de los estados. Recortado. `[highOrderManufacturedFDAImplicitPETScDistributed.C]` *(recortado; sigue en el fuente.)*

```
    void rkf45StateStep
    (
        const VmDriverNodes& vmNodes,
        const scalar tau,
        const scalar h,
        const scalar dt,
```

```
        const scalar u1,
        const scalar u2,
        const scalar u3,
        scalar& u1Fifth,
        scalar& u2Fifth,
        scalar& u3Fifth,
        const scalar stateODEAbsTol,
        const scalar stateODERelTol,
        scalar& err
    )
    {
        scalar k11 = 0.0, k12 = 0.0, k13 = 0.0;
        scalar k21 = 0.0, k22 = 0.0, k23 = 0.0;
        scalar k31 = 0.0, k32 = 0.0, k33 = 0.0;
        scalar k41 = 0.0, k42 = 0.0, k43 = 0.0;
        scalar k51 = 0.0, k52 = 0.0, k53 = 0.0;
        scalar k61 = 0.0, k62 = 0.0, k63 = 0.0;

        reactionRatesLinearVm
        (
            vmNodes, tau, dt,
            u1, u2, u3,
            k11, k12, k13
        );
        reactionRatesLinearVm
        (
            vmNodes, tau + h/5.0, dt,
            u1 + h*(1.0/5.0)*k11,
            u2 + h*(1.0/5.0)*k12,
            u3 + h*(1.0/5.0)*k13,
            k21, k22, k23
        );
        reactionRatesLinearVm
        (
            vmNodes, tau + 3.0*h/10.0, dt,
            u1 + h*((3.0/40.0)*k11 + (9.0/40.0)*k21),
            u2 + h*((3.0/40.0)*k12 + (9.0/40.0)*k22),
            u3 + h*((3.0/40.0)*k13 + (9.0/40.0)*k23),
            k31, k32, k33
        );
        reactionRatesLinearVm
        (
            vmNodes, tau + 3.0*h/5.0, dt,
            u1 + h*((3.0/10.0)*k11 - (9.0/10.0)*k21 + (6.0/5.0)*k31),
            u2 + h*((3.0/10.0)*k12 - (9.0/10.0)*k22 + (6.0/5.0)*k32),
            u3 + h*((3.0/10.0)*k13 - (9.0/10.0)*k23 + (6.0/5.0)*k33),
            k41, k42, k43
        );
        reactionRatesLinearVm
        (
            vmNodes, tau + h, dt,
            u1 + h*((-11.0/54.0)*k11 + (5.0/2.0)*k21 - (70.0/27.0)*k31 + (35.0/27.0)*k41),
            u2 + h*((-11.0/54.0)*k12 + (5.0/2.0)*k22 - (70.0/27.0)*k32 + (35.0/27.0)*k42),
            u3 + h*((-11.0/54.0)*k13 + (5.0/2.0)*k23 - (70.0/27.0)*k33 + (35.0/27.0)*k43),
```

Listing 32: El paso adaptativo. Recortado. `[highOrderManufacturedFDAImplicitPETScDistributed.C]` *(recortado; sigue en el fuente.)*

## 14. main(): montaje

`main()` arranca leyendo la malla y los diccionarios, construye los operadores de alto orden, y ensambla $M$ y $K$ **una sola vez** — son constantes, porque la no linealidad está en el término iónico y no en la difusión. Por eso en el desglose de tiempos el `setup` aparece separado del `loop`.

Es también donde se fija `gRowStart`, y donde viven los dos volcados de diagnóstico que la verificación usa, los dos detrás de una variable de entorno y gratis cuando están apagados: `CF_DUMP_K` escribe $K$ como tripletes (fila global, columna global, valor) y `CF_DUMP_STENCILS` escribe la membresía de los stencils. Son la base de M09, M10, M11 y M15.

Lo primero de `main()`, antes de cualquier asignación grande, son tres llamadas a `mallopt`, y no son cosmética: la construcción de los operadores corre unos **2.3 millones de factorizaciones QR** sobre una malla de tetraedros 3D con $N$=40 y p3, y sin ajustar el asignador glibc crece el heap en varios GB de fragmentos que no devuelve y la corrida muere por falta de memoria en una máquina de 16 GB. `M_ARENA_MAX=2` evita que glibc cree una arena por hilo de OpenMP — el montaje es serial, así que las arenas sólo inflan el RSS —, y los dos umbrales en 64 KB hacen que las asignaciones medianas pasen por `mmap` y que el relleno del heap vuelva al sistema operativo. Es el mismo techo de memoria que **M17** mide y el que explica por qué en 3D no se pasa de $N$=30.

```
2074 int main(int argc, char* argv[])
2075 {
2076 #ifdef __GLIBC__
2077     // Tighten the glibc allocator before any large allocation. The LRE
2078     // construction runs ~2.3M QR factorisations on a 3D unstructured tet
2079     // mesh with N=40 / p3; without these tunables glibc grows the heap by
2080     // several GB of unreclaimed fragments and the run OOMs on a 16 GB box.
2081     //   - M_ARENA_MAX=2: prevents glibc from creating ~64 arenas (one set
2082     //     per ODE OpenMP thread). LRE setup is serial; multiple arenas
2083     //     just inflate the RSS.
2084     //   - M_MMAP_THRESHOLD=64 KB: routes mid-sized allocations (Eigen
2085     //     temporaries during QR) through mmap so they are returned to
2086     //     the kernel individually when freed.
2087     //   - M_TRIM_THRESHOLD=64 KB: lets glibc release sbrk-heap padding
2088     //     back to the OS more aggressively.
2089     mallopt(M_ARENA_MAX, 2);
2090     mallopt(M_MMAP_THRESHOLD, 64*1024);
2091     mallopt(M_TRIM_THRESHOLD, 64*1024);
2092 #endif
2093
2094     #include "setRootCaseLists.H"
2095     PetscSession petscSession(argc, argv);
2096     #include "createTime.H"
2097     #include "createMesh.H"
```

Listing 33: El arranque. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
2077     // Tighten the glibc allocator before any large allocation. The LRE
2078     // construction runs ~2.3M QR factorisations on a 3D unstructured tet
2079     // mesh with N=40 / p3; without these tunables glibc grows the heap by
2080     // several GB of unreclaimed fragments and the run OOMs on a 16 GB box.
2081     //   - M_ARENA_MAX=2: prevents glibc from creating ~64 arenas (one set
2082     //     per ODE OpenMP thread). LRE setup is serial; multiple arenas
2083     //     just inflate the RSS.
2084     //   - M_MMAP_THRESHOLD=64 KB: routes mid-sized allocations (Eigen
2085     //     temporaries during QR) through mmap so they are returned to
2086     //     the kernel individually when freed.
2087     //   - M_TRIM_THRESHOLD=64 KB: lets glibc release sbrk-heap padding
2088     //     back to the OS more aggressively.
2089     mallopt(M_ARENA_MAX, 2);
2090     mallopt(M_MMAP_THRESHOLD, 64*1024);
2091     mallopt(M_TRIM_THRESHOLD, 64*1024);
```

Listing 34: El asignador, ajustado antes de la primera asignación grande. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
2337     // Publish this rank's global row offset for the
2338     // linear-solver wrappers. The matrices carry local rows and global columns,
2339     // so every PETSc insertion has to shift the row index by this amount. Zero
2340     // in serial, which is why the serial path is unaffected.
2341     initialiseGlobalCells(mesh);
2342
2343     Info<< "Distributed linear algebra: rows ["
2344         << gRowStart << ", " << gRowStart + mesh.nCells() << ") of "
2345         << LREInterp_Vm.globalCells().totalSize() << " global cells" << endl;
```

Listing 35: Dónde se fija el offset global de fila. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
486     void logMemoryCheckpoint(const word& stage)
487     {
488         Info<< "Memory checkpoint [" << stage << "]: peakRSS_MB = "
```

```
489             << currentPeakRSSMB() << nl;
490     }
```

Listing 36: Los checkpoints de memoria que M17 usa para ajustar el techo. [monodomainBase.H]

## 15. main(): el lazo temporal y los métodos no lineales

Cada paso resuelve un sistema no lineal, y hay tres métodos: **Picard** con relajación, **JFNK** — Newton sin jacobiano explícito, con el producto matriz-vector aproximado por diferencias — y **diagonalIion**, que linealiza la corriente iónica por su derivada diagonal. **M08** verifica que los tres lleguen al mismo lugar, y lo que exige no es una cota absoluta sino una *dirección de cambio*: que la dispersión entre los tres caiga en cada escalón de una escalera de `implicitTolerance`. Eso dice exactamente lo que se quiere afirmar — que lo que los separa es el piso del solve lineal y no una diferencia estructural — y no lleva ninguna constante. Su versión anterior derivaba una cota de `nonlinearVmTolerance`, y estaba indexada a una perilla que no gobernaba la cantidad que acotaba: apretarla cuatro órdenes no movía la discrepancia ni en el décimo dígito.

Ese test tenía además un punto ciego, y detrás había un defecto real en JFNK. Su lazo evalúa la convergencia dos veces por iteración, al tope y al final, y la del tope necesita los incrementos que acumuló la iteración *anterior*. Los recalculaba en el lugar contra una copia tomada unas líneas antes, con una sola re-evaluación del *mismo* punto en medio, así que valían cero por construcción: cuatro de las cinco tolerancias comparaban cero contra un número positivo y el test se reducía al residual acoplado. Consecuencia medida en 3-D: en 24 de 50 pasos JFNK devolvía la extrapolación inicial sin dar un paso de Newton y con cero llamadas al KSP. Se arregló guardando los incrementos medidos al final de cada iteración; `max coupled_relL2` cayó de 9.493e-09 a 5.655e-16 y Picard y `diagonalIion` quedaron bit a bit idénticos.

Al agotar las iteraciones sin converger hay dos salidas posibles, y **los tres métodos las eligen ahora con la misma clave**: `false` descarta el paso y vuelve a los valores del inicio, `true` se queda con el iterado y lo dice. Antes Picard tenía su propia rama que aceptaba el último iterado *siempre*, así que el mismo solver tenía dos semánticas de error según el método. Se eliminó esa rama en vez de agregarle un rollback propio: las otras dos ya implementan las dos salidas, así que Picard cae solo en la correcta.

Medido forzando la no convergencia con `nonlinearIterations 1` y tolerancias en 1e-30, que es la única forma de hacerlo disparar — en los barridos de convergencia no ocurre nunca: con `false` el error final es **4.998e-04**, que es el de no haber avanzado el paso, y con `true` es **5.023e-06**. El rollback efectivamente revierte.

Un nombre de método inválido es `FatalError` y no un silencioso repliegue sobre Picard, que es lo que hacía el solver electro hasta que **E18** lo señaló.

**Un solo criterio de parada para los tres métodos (2026-10).** Picard paraba antes sólo con los incrementos de $V_m$ y de los estados, mientras JFNK y `diagonalIion` exigían además el residuo acoplado: una comparación de métodos comparaba, en parte, reglas de parada. Ahora los tres llaman a `nonlinearIterationExit` (`monodomainBase.H`, la misma del electro): incrementos de $V_m$ e $I_{ion}$ — y de los estados si `nonlinearRequireStatesConvergence` — y residuo acoplado.

Con una salida más, la de **piso**: el residuo acoplado puede quedarse quieto por encima de `nonlinearTolerance` (el residuo verdadero que deja un Krylov precondicionado, o redondeo en un sistema rígido grande), y cada iteración extra repite el mismo solve hasta el tope. Se encontró en este solver: Picard, `ilu`, $\Delta t = 2 \times 10^{-3}$, 2000 iteraciones idénticas por paso con un residuo de $3{,}66 \times 10^{-12}$ contra una tolerancia de $10^{-12}$. Si los incrementos ya pasan y el residuo no bajó desde la iteración anterior, el iterado está tan convergido como el operador permite: se para, se cuenta y se avisa en el log. Los incrementos nunca se aflojan, así que el peor caso es el criterio viejo de Picard.

**La condición de parada del lazo tenía un error, y era invisible.** El tiempo acumulado es una suma en punto flotante de $\Delta t$, y puede caer por debajo de `endTime` *por más que* `SMALL`: 160 pasos de $3{,}2 \times 10^{-3}$ suman 0,51199999999999868, a $1{,}3 \times 10^{-15}$ de 0,512. Con la condición vieja, `endTime - SMALL`, el lazo daba un paso de más y la corrida terminaba en 0,5152 sin avisar nada. Eso no es cosmético para un estudio temporal: los campos que se comparan entre corridas con distinto $\Delta t$ dejan de ser del mismo instante. Ahora la condición es `endTime` $- \Delta t/2$, exacta porque acá $\Delta t$ es constante. Lo encontró la guarda de un test de la suite, no una revisión del código. El solver electro, que admite $\Delta t$ variable, usa su propia tolerancia de $10^{-3}\,\Delta t$.

```
    NonlinearExit nonlinearIterationExit
    (
        const label corr,
        const label minIterations,
        const scalar coupledResidual,
        const scalar coupledResidualPrevIter,
        const scalar coupledTolerance,
        const scalar VmIncrement,
        const scalar VmTolerance,
        const scalar IionIncrement,
        const scalar IionTolerance,
        const bool requireStates,
        const scalar maxStateIncrement,
        const scalar statesTolerance
    )
    {
        const bool incrementsPass =
            corr + 1 >= minIterations
         && VmIncrement <= VmTolerance
         && IionIncrement <= IionTolerance
         && (!requireStates || maxStateIncrement <= statesTolerance);

        if (!incrementsPass)
        {
            return NonlinearExit::Continue;
        }
        if (coupledResidual <= coupledTolerance)
        {
            return NonlinearExit::Converged;
        }
        if (coupledResidual >= coupledResidualPrevIter)
        {
            return NonlinearExit::ResidualFloor;
        }
        return NonlinearExit::Continue;
    }
```

Listing 37: El criterio común, con la salida de piso. [monodomainBase.H]

```
    const bool usePicard = nonlinearMethod == "Picard";
    const bool useDiagonalIion = nonlinearMethod == "diagonalIion";
    const bool useJFNK = nonlinearMethod == "JFNK";

    if (!usePicard && !useDiagonalIion && !useJFNK)
    {
        FatalErrorInFunction
            << "Unknown nonlinearMethod '" << nonlinearMethod << "'. "
            << "Valid options are Picard, JFNK and diagonalIion (exact"
            << " spelling, case-sensitive)."
            << exit(FatalError);
    }
```

Listing 38: El despacho de método no lineal, y el rechazo de un nombre inválido. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
        // Picard used to have its own branch here
        // that accepted the last iterate unconditionally, so the same solver
        // had two error semantics depending on the method. All three now obey
        // nonlinearAcceptUnconverged: false rolls the step back, true keeps the
        // iterate and says so.
        if (!nonlinearConverged && !nonlinearAcceptUnconverged)
        {
            WarningInFunction
```

```
                << "Nonlinear solver did not converge at t = " << t
                << " after " << nonlinearIters << " iterations."
                << " Rolling back Vm, Iion and states to the values"
                << " at the beginning of the time step." << nl
                << "Set 'nonlinearAcceptUnconverged true;' in"
                << " spatialIntegrationProperties to accept unconverged"
                << " iterates instead." << endl;

            VmGuess.primitiveFieldRef() = VmOld.primitiveField();
            VmGuess.correctBoundaryConditions();
            u1Guess.primitiveFieldRef() = u1Old.primitiveField();
            u2Guess.primitiveFieldRef() = u2Old.primitiveField();
            u3Guess.primitiveFieldRef() = u3Old.primitiveField();
            updateStateBoundaryValues(u1Guess, u2Guess, u3Guess, t, dim);
            u1Guess.correctBoundaryConditions();
            u2Guess.correctBoundaryConditions();
            u3Guess.correctBoundaryConditions();
            IionGuess.primitiveFieldRef() = Iion.primitiveField();
            IionGuess.correctBoundaryConditions();
            nonlinearRolledBack = true;
```

Listing 39: Las dos salidas al agotar las iteraciones, ahora comunes a los tres métodos. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
    // Stop within half a step of endTime, not within
    // SMALL. The accumulated time is a floating-point sum of dt, and it can land
    // just BELOW endTime by more than SMALL (1e-15): 160 steps of 3.2e-3 sum to
    // 0.51199999999999868, which is 1.3e-15 short of 0.512. The old test then
    // took one extra step and the run finished at 0.5152 with no warning - so a
    // field or an error "at endTime" belonged to a different instant, and a
    // temporal-order study comparing runs at different dt compared fields at
    // different times. Half a step is exact because dt is constant here, and it
    // leaves every run that was already correct with the same step count:
    // checked across 159 151 existing summaries, none of which had an extra step.
    // Nonlinear iterations that stopped at the residual floor instead of the
    // tolerance (nonlinearIterationExit, monodomainBase.H).
    label nResidualFloorExits = 0;

    while (runTime.value() < runTime.endTime().value() - 0.5*dt)
    {
```

Listing 40: El lazo que ya no da un paso de más, y por qué. [highOrderManufacturedFDAImplicitPETScDistributed.C]

```
            // The increments measured at the END of the previous corr, which are
            // what the check at the top of the next one is entitled to look at.
            //
            // That check used to recompute them against a copy taken two lines
            // further up, with only residualFor(x, true) in between - and that
            // call re-evaluates the same x, so the "increments" were the
            // difference between two evaluations of one point, i.e. identically
            // zero. Four of the five tolerance terms therefore compared zero
            // against a positive number and could never fail: the test reduced
            // to `newtonResidual <= nonlinearTolerance` alone, and JFNK returned
            // the jfnkInitGuessOrder extrapolation without taking a single
            // Newton step whenever that extrapolation happened to land under the
            // coupled tolerance. Measured on M08 in 3-D: 24 of 50 timesteps
            // exited that way, 6.3e-08 from the solution the other two
            // nonlinear methods agree on, and no tolerance the user could set
            // made any difference - only nonlinearTolerance did.
            //
            // GREAT on entry means the first pass can never exit there, so at
            // least one Newton step is always taken. Verified by construction
            // against minNonlinearIterations = 2, which forces the same thing
            // from the outside and closes the gap to 2.0e-11.
            scalar VmIncrPrevIter = GREAT;
            scalar u1IncrPrevIter = GREAT;
            scalar u2IncrPrevIter = GREAT;
            scalar u3IncrPrevIter = GREAT;
            scalar IionIncrPrevIter = GREAT;
```

Listing 41: Por qué los incrementos del chequeo del tope de JFNK viven fuera del lazo: recalculados adentro valían cero y mataban cuatro tolerancias. [highOrderManufacturedFDAImplicitPETScDistributed.C]

# 16. Índice de funciones

Todas las funciones, archivo por archivo y en orden de aparición: primero las del solver, después las de los headers compartidos de `src/monodomainCommon`. Las secciones anteriores recorren las principales; ésta es el mapa completo.

`highOrderManufacturedFDAImplicitPETScDistributed.C`

| línea | función | líneas |
|---|---|---|
| 164 | `computeF` | 25 |
| 192 | `computeG` | 21 |
| 217 | `exactVm` | 4 |
| 224 | `exactU1` | 4 |
| 230 | `exactU2` | 4 |
| 237 | `exactU3` | 4 |
| 244 | `exactFieldValue` | 20 |
| 268 | `reactionRates` | 18 |
| 291 | `ionicCurrentPDE` | 15 |
| 311 | `vmSourcePDE` | 13 |
| 329 | `computeBeta` | 24 |
| 356 | `estimatedN` | 8 |
| 369 | `characteristicDx` | 35 |
| 409 | `volumeWeightedL1` | 7 |
| 419 | `volumeWeightedL2` | 7 |
| 428 | `linfNorm` | 5 |
| 437 | `cellErrorNorm` | 4 |
| 442 | `computeFieldErrorSummary` | 9 |
| 454 | `sourceToEigVec` | 4 |
| 464 | `appendTetQuadraturePointsAndWeights` | 58 |
| 527 | `cellQuadraturePointsAndWeights` | 71 |
| 602 | `updateStateBoundaryValues` | 36 |
| 641 | `fillExactFields` | 60 |
| 704 | `computeCellCentredIion` | 34 |
| 740 | `computeCellCentredVmSource` | 34 |
| 778 | `reconstructStatesAtIionIntegrationPoints` | 51 |
| 833 | `averageIntegrationPointFieldToCells` | 32 |
| 869 | `initialiseIionIntegrationPointStates` | 26 |
| 899 | `computeIionFromIntegrationPoints` | 32 |
| 933 | `computeVmSourceFromIntegrationPoints` | 32 |
| 966 | `reactionRatesLinearVm` | 22 |
| 991 | `rk4StateStep` | 54 |
| 1050 | `rkf45StateStep` | 92 |
| 1147 | `advanceStateODE` | 125 |
| 1275 | `updateIntegrationPointStatesODE` | 57 |
| 1333 | `updateStateFieldsODE` | 72 |
| 1406 | `computeGaussReconstructedError` | 109 |
| 1516 | `cellAsReconstructionError` | 14 |
| 1531 | `computeIntegrationPointStateError` | 65 |
| 2074 | `main` | 3355 |

monodomainAssembly.H

| línea | función | líneas |
|---|---|---|
| 30 | exchangeCoupledFaceRows | 78 |
| 113 | orthogonalDiffusionCoeff | 14 |
| 130 | normalDiffusivity | 4 |
| 140 | stabilisationFaceCoeff | 17 |
| 193 | nonOrthFluxVector | 30 |
| 233 | computeHighOrderLaplacian | 80 |
| 317 | addDiagonalToMatrix | 18 |
| 338 | assembleDiagonalMassMatrix | 31 |
| 373 | addCellGradientDotCoeffs | 58 |
| 434 | buildCellGradientDotRow | 26 |
| 477 | cellTaylorEntryCoeff | 32 |
| 513 | buildCellTaylorExtrapolationRow | 29 |
| 550 | addCellTaylorExtrapolationCoeffs | 57 |
| 619 | addRhieChowInternalJumpCoeffs | 34 |
| 660 | addRhieChowCoupledJumpCoeffs | 38 |
| 706 | assembleConsistentMassMatrixHO | 237 |
| 968 | assembleStandardOrthogonalStiffnessMatrix | 412 |
| 1400 | assembleHighOrderStiffnessMatrix | 445 |

monodomainBase.H

| línea | función | líneas |
|---|---|---|
| 33 | globalRowStart | 14 |
| 55 | gCol | 4 |
| 61 | gNCols | 4 |
| 70 | initialiseGlobalCells | 6 |
| 87 | requireOneOf | 31 |
| 126 | gSquaredNorm | 6 |
| 135 | gNorm | 4 |
| 143 | gDot | 6 |
| 153 | fieldToEigVec | 12 |
| 169 | eigVecToField | 9 |
| 181 | relativeL2Norm | 5 |
| 191 | addTripletIfNeeded | 13 |
| 207 | addRemoteRowCoeffs | 19 |
| 239 | clampPhysical | 13 |
| 258 | relativeL2Difference | 29 |
| 301 | reportOperatorFingerprint | 35 |
| 357 | nonlinearIterationExit | 36 |
| 396 | nonlinearIterationStops | 18 |
| 468 | currentPeakRSSMB | 14 |
| 486 | logMemoryCheckpoint | 5 |
| 494 | normalizedMassMatrixType | 5 |
| 505 | memoryOptimizationIsEffective | 22 |

monodomainKrylov.H

| línea | función | líneas |
|---|---|---|
| 29 | `diagonalBlock` | 23 |
| 66 | `solveGMRES` | 142 |
| 315 | `beginStep` | 4 |
| 329 | `update` | 92 |
| 423 | `apply` | 25 |
| 483 | `solveJfnkLinearSystem` | 44 |
| 569 | `solveSparseSystemEigen` | 94 |
| 672 | `solveSparseSystem` | 101 |

`monodomainPetsc.H`

| línea | función | líneas |
|---|---|---|
| 20 | `checkPetscError` | 10 |
| 32 | `copyPetscVecToEigVec` | 16 |
| 60 | `buildDistributedMat` | 85 |
| 177 | `clear` | 15 |
| 193 | `reset` | 19 |
| 306 | `setPetscOptionsPrefix` | 11 |
| 327 | `usesPetscBackend` | 5 |
| 344 | `resolvePetscKspPc` | 18 |
| 373 | `petscKspTypeForLinearSolver` | 12 |
| 395 | `petscParallelPcTypeName` | 13 |
| 411 | `isGmresType` | 4 |
| 428 | `copyEigVecToPetscVec` | 12 |
| 706 | `updateValues` | 69 |
| 776 | `solve` | 57 |
| 846 | `petscShellMatMult` | 17 |
| 866 | `petscShellPCApply` | 17 |
| 954 | `initialise` | 172 |

`monodomainReconstruction.H`

| línea | función | líneas |
|---|---|---|
| 17 | `quadraticForm` | 10 |
| 35 | `reconstructFromTaylor` | 24 |
| 99 | `reconstructCellFieldsAtIntegrationPoints` | 249 |
| 355 | `reconstructVmAtIionIntegrationPoints` | 73 |
| 436 | `reconstructVmLevelAtIionIntegrationPoints` | 29 |

`monodomainTimeSchemes.H`

| línea | función | líneas |
|---|---|---|
| 77 | `esdirk3Tableau` | 37 |
| 117 | `timeSchemeCoeffs` | 71 |
| 217 | `interpolateVmDriver` | 31 |